# Model Compression for LLMs

**Ka Kit Li** K.Li@campus.lmu.de
*Department of Statistics*
*Ludwig Maximilian University of Munich, Munich, Germany*

## Abstract

The rapid advancement of large language models (LLMs) has led to superior performance in a wide range of natural language processing tasks, yet their immense size and computational demands present significant deployment challenges on resource-constrained devices. Training these large models from scratch is also prohibitively expensive and time-consuming, necessitating more efficient methods for creating compact, deployable models. In this paper, we review and analyze the Minitron model, a state-of-the-art approach to LLM compression that leverages a novel pipeline of iterative structured pruning and knowledge distillation. The theoretical background and recent developments of model compression with neural architecture search (NAS) in LLM-scale will also be investigated. Minitron starts with a large pretrained model, computes activation-based importance scores, and uses a lightweight NAS to identify and prune redundant components. The resulting smaller models are then retrained through distillation to recover performance. This iterative process effectively creates a family of highly efficient models. The Minitron framework demonstrates substantial cost savings, reducing overall training expenditure by 44.5% and achieving a 97.5% cost reduction for the smaller 4B and 8B variants compared to training from scratch. The resulting models not only outperform similarly sized baselines but also achieve performance on par with larger community models while being trained on a fraction of the data. The generalizability of this method has been confirmed by its successful application to other foundational models, extending its impact to various real-world applications. This work provides a comprehensive overview of a scalable and effective strategy for bridging the gap between LLM capabilities and practical deployment.

**Keywords:** neural architecture search, model compression, pruning, knowledge distillation, large language models

## 1 Introduction

As mentioned in Wang et al. (2024b), transformer-based large language models (LLMs) have achieved remarkable advancements in natural language processing, exhibiting superior performance across a wide range of tasks. These models, including ChatGPT, LLaMA, and Claude, have demonstrated exceptional generalization capabilities and exhibit *emergent abilities*—such as multi-step reasoning and instruction following—that are typically absent in smaller models. The unprecedented performance of LLMs has been further corroborated by recent developments, such as the LLaMA-3.1 model with 405 billion parameters, which substantially outperforms its smaller counterpart in dialogue, reasoning, and programming tasks (Wang et al., 2024a). This trend aligns with the neural scaling law, which suggests that larger models trained with more data over longer durations tend to achieve improved performance across tasks.

©2025 Ka Kit Li.
License: CC-BY 4.0, see https://creativecommons.org/licenses/by/4.0/.

Despite these strengths, LLMs also pose significant deployment challenges. As emphasized in Wang et al. (2024a), the extensive parameter count and computational requirements hinder real-time inference and limit applicability on resource-constrained devices. For instance, LLaMA-3.1 405B, trained over 54 days using 16k NVIDIA H100 GPUs, demands over 200 GB of GPU memory even with int4 quantization and introduces latency unsuitable for real-time applications. Furthermore, the reliance on cloud-based APIs for inference raises privacy concerns, particularly in sensitive domains like healthcare and finance. On-device deployment is rendered impractical due to hardware limitations, and fine-tuning such models for domain-specific tasks remains computationally intensive. Moreover, in many real-world scenarios, the full general-purpose capabilities of LLMs are superfluous, and they may even underperform when compared to smaller models specifically tuned for narrow domains.

Given these limitations, there is a growing demand for more efficient, smaller-scale language models that can support real-time, private, and low-latency inference on edge devices. As outlined in Muralidharan et al. (2024), model providers often address this by training a family of LLMs from scratch, each varying in size to accommodate different deployment needs. Examples include the LLaMA-2 family, with models ranging from 7B to 70B parameters, and the Pythia suite, which spans models from 80M to 12B parameters. However, this approach is prohibitively expensive: training a single 10B parameter model on 8 trillion tokens using float32 weights may consume over 37 GB of memory and incur monetary and temporal costs exceeding one million USD and a week of computation, respectively.

In contrast to training multiple models from scratch, model compression techniques—such as pruning, quantization, and knowledge distillation—offer a more practical and cost-effective strategy for obtaining compact language models suitable for deployment on edge devices. As noted by Muralidharan et al. (2024), compression assumes access to a pretrained model and focuses on reducing memory and computational overhead during inference. These methods significantly reduce the model footprint while preserving most of the original capabilities, making them highly attractive for on-device inference under limited hardware budgets.

### 1.1 Small Language Models

As discussed in Wang et al. (2024a), small language models (SLMs) are gaining increasing prominence due to their advantages in low-latency inference, cost-effective deployment, efficient training, and ease of customization. Unlike LLMs, which require significant memory and computational resources, SLMs are better suited for resource-constrained environments and applications demanding localized data processing, minimal latency, and enhanced privacy. Their compact architecture enables deployment on low-power devices such as smartphones and edge devices, facilitating improved response times, energy efficiency, and reduced carbon emissions. For instance, recent models such as LLaMA 3.2 (1B- and 3B-parameters versions) enable on-device execution of prompts and responses, thereby preserving user privacy by keeping sensitive data—such as personal messages, health records, or proprietary business information—local to the device.

Owing to their smaller parameter count, SLMs offer significant savings in both training and inference, with reduced storage and memory requirements. These characteristics make them ideal for specialized domains where general-purpose LLMs may be computationally prohibitive or less effective. Furthermore, the ability to fine-tune SLMs with modest resources enables

rapid customization for domain-specific tasks, enhancing their adaptability and practical utility in real-world applications. As a result, major technology companies have begun investing heavily in SLM development, as exemplified by Microsoft's Phi-3 model family, which includes variants small enough for smartphone deployment.

Despite their growing adoption, the definition of what constitutes an SLM remains inconsistent across the literature. As noted in Wang et al. (2024a), some studies classify models with fewer than one billion parameters as SLMs, while others adopt a relative definition based on comparisons with larger models. There is also variation depending on deployment targets: for mobile devices with approximately 6 GB of memory, sub-billion parameter models are typically favored, whereas other settings consider models up to 10 billion parameters to be "small," provided they lack emergent capabilities. This lack of consensus motivates the need for a more standardized definition.

To address this ambiguity, we adopt a task- and resource-oriented definition of SLMs from Wang et al. (2024a). Specifically, SLMs are defined as language models whose parameter size lies between: (1) the minimum threshold required to exhibit emergent abilities for a given specialized task, and (2) the maximum size that can be deployed under predefined resource constraints, such as memory or power limitations. This definition captures the trade-off between model capability and deployability, and offers a practical framework for guiding SLM development across varying application scenarios.

### 1.2 Main Research Paper

One prominent example of SLM is *Minitron*, introduced in the research paper titled *"Compact Language Models via Pruning and Knowledge Distillation"* by Muralidharan et al. (2024), authored by a team of researchers at NVIDIA. This work provides a comprehensive empirical study on structured pruning and retraining strategies applied to large language models. In particular, the paper offers valuable insights into the optimal ordering of pruning operations, the interplay between different pruning axes, and efficient retraining methods designed to minimize data requirements while maintaining model performance.

The remainder of this paper is organized as follows: Section 2 presents necessary background information to contextualize the following discussions. Then, the model compression methods adopted by Minitron will be explored in length. Sections 3, 4, and 5 provide detailed discussions on pruning, neural architecture search, and knowledge distillation, respectively. In Section 6, we review experimental results associated with Minitron, highlighting its efficiency and effectiveness. Finally, Section 7 concludes the paper and outlines directions for future work.

## 2 Background

In this section, we provide an overview of the Minitron model along with related models in the area of model compression, both prior to and following its introduction. We then present a detailed discussion of common compression techniques developed to address the challenges of deploying LLMs in resource-constrained environments.

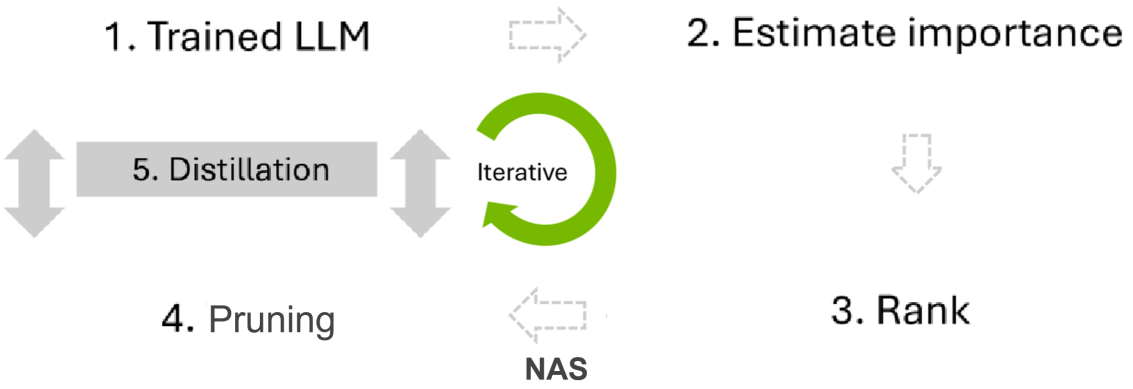



Figure 1: Simplified overview of iterative pruning and distillation approach in Minitron. NAS stands for neural architecture search. More details in Section 3.2.

### 2.1 Minitron

*Minitron* is a compact language model developed by Muralidharan et al. (2024), a group of researchers at NVIDIA, that exemplifies an efficient and scalable approach to constructing SLMs. As shown in Figure 1, the model is derived through a carefully designed combination of importance ranking, structured pruning and knowledge distillation techniques applied to a larger pretrained model (Nemotron-4 15B). Minitron leverages multiple axes of structured pruning—including neuron, attention head, embedding, and depth pruning—followed by data-efficient retraining. The resulting models achieve competitive accuracy while dramatically reducing the compute cost of training and inference, positioning Minitron as a practical and high-performing alternative to models trained entirely from scratch.

Crucially, Minitron represents the first LLM-scale implementation that systematically integrates structured pruning with knowledge distillation to produce compressed models using minimal retraining data, while requiring only forward propagation for importance computation. Previous approaches in structured pruning often overlooked the retraining phase or relied on resource-intensive full retraining. In contrast, Minitron demonstrates that high-performing SLMs can be obtained by retraining on less than 3% of the original dataset. This enables the derivation of smaller models—specifically, 8B and 4B parameter versions—from an existing 15B model, avoiding the need to train each variant from scratch. As a result, this approach significantly reduces computational burden (1.8× less FLOPs) while preserving generalization and task performance.

The cost savings realized through the Minitron framework are substantial. Training three models (Nemotron-4 15B, Minitron 8B, and Minitron 4B) using this compression pipeline reduces the overall training expenditure from approximately USD 2.54 million to USD 1.41 million—a 44.5% reduction. When considering only the two smaller models (8B and 4B), the savings are even more pronounced, with training costs dropping by 97.5% (from USD 1.16 million to just USD 28,993). This efficiency is primarily attributed to the dramatic reduction in training tokens, enabled by reusing the pretrained 15B model and applying data-efficient

| | Trained from scratch | | Minitron | |
|---|---|---|---|---|
| **Model Size** | **FLOPs ($10^{23}$)** | **Cost (USD)** | **FLOPs ($10^{23}$)** | **Cost (USD)** |
| 15B | 7.49 | 1,379,105 | 7.49 | 1,379,105 |
| 8B | 4.25 | 783,582 | 0.11 | 19,590 |
| 4B | 2.04 | 376,119 | 0.05 | 9,403 |
| **Total** | 13.78 | 2,538,806 | 7.65 | 1,408,097 |

Table 1: Comparison of computational and monetary training costs for the training from scratch approach and Minitron approach, given that the same Nemotron-4 15B model is adopted as base model.

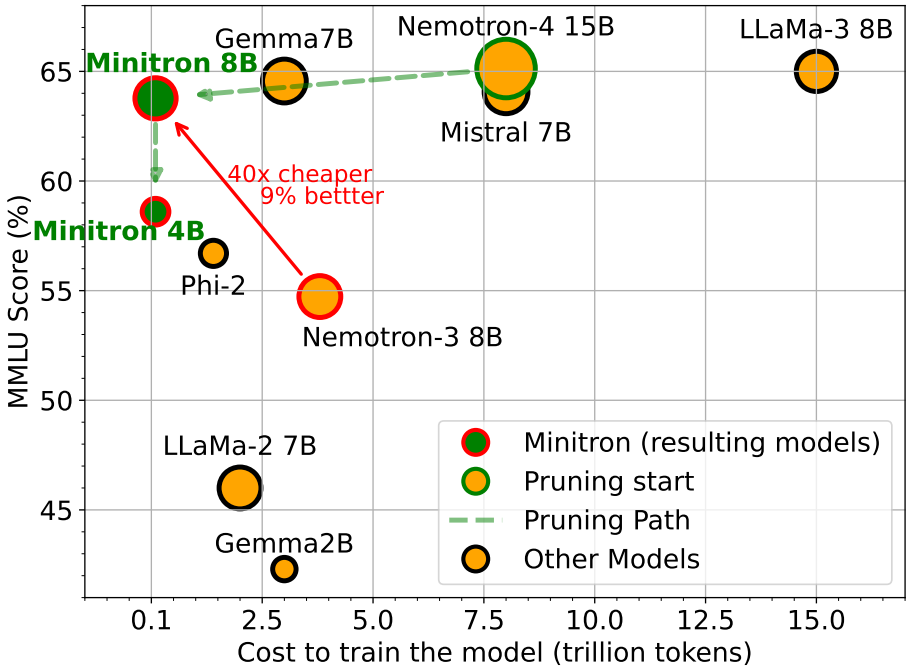



Figure 2: Results for Minitron, assuming cost is proportional to the number of training tokens used. More information for MMLU score is shown in Appendix B. Source: Muralidharan et al. (2024)

retraining strategies. Further details and cost calculation assumptions are shown in Table 1 and Appendix A.

Empirically, Minitron models exhibit strong performance across standard language modeling benchmarks (see Figure 2). The Minitron-8B model achieves higher accuracy than Nemotron-3 8B and LLaMA-2 7B, despite being trained on 40× fewer tokens. Moreover, it performs on par with widely-used community models such as Mistral-7B, Gemma-7B, and LLaMA-3 8B. The Minitron-4B variant also outperforms comparably sized models

like Gemma2 and Phi-2. Notably, Minitron models outperform state-of-the-art pruning techniques in terms of both compression ratio and downstream accuracy, making them a compelling benchmark for efficient model construction. More experiment results are shown in Section 6.2.

The Minitron model weights and supporting resources have been publicly released on Hugging Face [1], with example code and documentation available on GitHub [2], facilitating further research and reproducibility.

### 2.2 Minitron-Related Models

The development of Minitron builds on prior advances in structured pruning, particularly those exemplified by earlier works such as *Sheared-LLaMA*. As discussed in Xia et al. (2024), Sheared-LLaMA introduced a novel framework for reducing LLMs through targeted structured pruning and continued pre-training. The authors pruned LLaMA2-7B down to compact variants with 1.3B and 2.7B parameters by removing layers, attention heads, and hidden dimensions in a systematic, end-to-end manner. Their approach also incorporated dynamic batch loading to improve training efficiency across domains. Despite using only 3% of the computational budget required for training from scratch, Sheared-LLaMA achieved performance that outperformed other similarly sized models, including Pythia, INCITE, OpenLLaMA, and TinyLLaMA. This work demonstrated that structured pruning of pretrained models offers a cost-effective path to building competitive small-scale LLMs.

The foundation of the Minitron model is *Nemotron-4 15B* (Parmar et al., 2024), a decoder-only transformer trained on 8 trillion tokens spanning English, 53 natural languages, and 43 programming languages. Nemotron-4 15B achieves state-of-the-art performance among models of similar size, outperforming larger and specialized multilingual models in many benchmarks. Its exceptional multilingual and coding capabilities make it a strong candidate as a base model for compression. These strengths enable Minitron to inherit a broad skill set even after pruning and distillation.

Building upon this foundation, Minitron applies structured pruning combined with knowledge distillation to derive smaller, efficient models from Nemotron-4 15B. The effectiveness of this compression strategy extends beyond the original Minitron pipeline. As demonstrated by Sreenivas et al. (2024), the same methodology was successfully applied to compress Mistral-NeMo-12B and LLaMA-3.1-8B into 8B and 4B models, respectively. This work introduced an additional teacher correction phase to adapt the teacher model to domain-specific data using lightweight fine-tuning prior to distillation. The result is a family of high-performing, instruction-aligned models, including the state-of-the-art Mistral-NeMo-Minitron-8B, which exhibits strong performance across instruction following, role-play, coding, and function-calling tasks. These findings confirm the generalizability of Minitron's compression techniques across diverse model architectures and deployment scenarios.

One notable deployment of a Minitron-derived model is *Nemotron-Mini-4B-Instruct*, optimized for on-device inference and real-world applications. This model is a distilled and pruned version of Nemotron-4 15B, fine-tuned for tasks such as role-playing dialogue, retrieval-augmented generation (RAG), and function calling. Further enhanced with int4

1. Minitron Collection on HuggingFace
2. `https://github.com/NVlabs/Minitron`

quantization, the model achieves a memory footprint of approximately 2 GB VRAM and supports a context length of 4,096 tokens. At Gamescom 2024, NVIDIA announced this model as part of the ACE suite (digital human technologies powered by generative AI), introducing the first on-device small language model tailored to game development for improving the conversation abilities of game characters [3]. Integrated into Seasun Games' *Mecha BREAK*, Nemotron-Mini-4B-Instruct enables dynamic character interactions and enhances player immersion using NVIDIA's GeForce RTX AI PCs. The model is available to developers as part of NVIDIA NIM, a set of deployable microservices supporting both cloud and local inference.

Together, these developments illustrate the growing impact of Minitron and its associated compression methodology—not only in research but also in practical, high-impact deployments. From foundational advancements in pruning to scalable application across different model backbones, and ultimately to commercial deployment in interactive environments.

### 2.3 Model Compression

As highlighted in Wang et al. (2024b), model compression plays a critical role in reducing the computational and memory footprint of large language models during inference, thereby enabling their deployment on devices with limited resources. From an algorithmic perspective, the compression and acceleration of LLMs can be categorized into several key approaches: quantization, pruning, knowledge distillation, compact architecture design, and dynamic networks. While these methods are broadly applicable to smaller models as well, LLMs introduce distinct challenges that necessitate new strategies.

First, compression algorithms for LLMs often require fine-tuning or retraining the model post-compression to recover the performance loss. However, the high computational and financial cost of fine-tuning large models has led to increased interest in tuning-free or low-resource fine-tuning methods. Second, unlike models designed for single-task performance, LLMs prioritize versatility and generalization across diverse tasks and domains. As a result, compression techniques must ensure that these broad capabilities—including emergent abilities—are preserved.

Below, we briefly describe the main model compression techniques:

- **Quantization:** This method reduces model size by converting high-precision floating-point weights (for example, float32) into lower-bit representations, such as int8 or float16. This leads to lower memory consumption, improved inference speed, and potentially greater parallelism.

- **Pruning:** Pruning removes redundant or less important components of a model, such as neurons, heads, or even entire layers. Structured pruning strategies are especially valuable in LLMs for reducing computational overhead without significant degradation in performance.

- **Knowledge Distillation:** In this approach, a large, pretrained model (the teacher) transfers its knowledge to a smaller model (the student), which is trained to replicate

3. NVIDIA technical blog: Deploy the First On-Device Small Language Model for Improved Game Character Roleplay

the teacher's output. This enables the student model to retain much of the teacher's capability while being more efficient in size and speed.

- **Compact Architecture Design:** This involves designing lightweight model architectures that replace computationally expensive components—such as self-attention mechanisms in Transformers—with more efficient alternatives. These architectures are often optimized specifically for mobile or edge deployment.
- **Dynamic Networks:** Dynamic models adapt their structure at inference time based on the input. A prominent example is the mixture of experts (MoE) framework, in which only a subset of the network (specialized experts) is activated per input, reducing the average computation per inference step.

In practice, these techniques are often combined to achieve higher compression rates and greater inference efficiency. Despite significant advances, compressing LLMs remains a challenging task. Consequently, a new class of compression techniques tailored specifically for LLMs has emerged, focusing on maintaining generalization, minimizing fine-tuning requirements, and enabling deployment across diverse platforms, including edge and mobile environments.

## 3 Pruning

Pruning is a well-established technique for neural network compression and acceleration, wherein non-essential weights or structural components are removed while maintaining the model's original performance as closely as possible. As described in Wang et al. (2024b), pruning has demonstrated significant success in convolutional neural networks (CNNs). However, its efficacy in LLMs has proven more limited when compared to other techniques such as quantization or knowledge distillation. This diminished effectiveness largely stems from the fine-tuning process: fine-tuning pruned LLMs is computationally expensive due to the large number of parameters involved, making it challenging to fully realize the benefits of pruning. Despite these challenges, pruning remains an important strategy in the model compression toolkit, warranting further refinement to enhance its utility for LLMs.

### 3.1 Pruning Units and Methodologies

A fundamental aspect of pruning involves determining the appropriate granularity of the elements to be removed—referred to as *pruning units*. These units may include individual weights, neurons, attention heads, layers, and other network components. Based on the level of granularity, pruning techniques are broadly categorized into two types: *unstructured pruning* and *structured pruning*.

In unstructured pruning, the pruning units are individual weights. Selected weights are set to zero, resulting in high sparsity and often minimal degradation in model performance. However, because the resulting sparsity pattern is irregular, the computational speedup during inference is negligible—sparse weight matrices often behave similarly to dense ones from a performance perspective.

Structured pruning, in contrast, removes larger, coherent structures within the network, such as neurons, attention heads, or entire layers. This approach enables meaningful inference

acceleration, as the resulting models are better aligned with hardware efficiencies. However, structured pruning typically imposes stricter constraints and may lead to greater performance degradation, necessitating lower sparsity ratios to avoid collapsing model performance.

Pruning can be mathematically formalized using binary masks applied to weights or structural components. In unstructured pruning, a binary mask $z$ is applied element-wise to the weight matrix $w$, and the loss optimization problem is defined as:

$$\min_{w,z} \mathcal{L}(w \odot z; \mathcal{D}) = \min_{w,z} \frac{1}{N} \sum_{i=1}^{N} \ell(w \odot z; (x_i, y_i)) \quad \text{s.t.} \quad \|z\|_0 \leq t,$$

where $\odot$ denotes Hadamard product, $w = \{w_1, w_2, \ldots, w_M\}$ represents the model parameters, $\mathcal{D}$ is the dataset with $N$ input-output pairs $(x_i, y_i)$, and $t$ defines the non-sparsity threshold.

For structured pruning, the binary mask $z$ operates on structured components $s = \{s_1, s_2, \ldots, s_K\}$, and the problem is reformulated as:

$$\min_{w,z} \mathcal{L}(s \odot z; \mathcal{D}) = \min_{w,z} \frac{1}{N} \sum_{i=1}^{N} \ell(s \odot z; (x_i, y_i)) \quad \text{s.t.} \quad f(z; s) \leq t,$$

where $f(\cdot)$ is a function that computes the non-sparsity ratio based on the applied binary mask and the structural components. This formulation supports optimization over structured units such as attention heads, feed-forward layers, or hidden dimensions, aligning the pruning process with hardware-friendly design principles.

### 3.2 Iterative Pruning and Distillation in Minitron

The Minitron model is developed through a pipeline of iterative pruning and knowledge distillation, as outlined by Muralidharan et al. (2024). As shown in Figure 3, the process follows a multi-stage procedure in which each stage produces a progressively smaller and more efficient model using activation-based importance estimation, structured pruning, and distillation from a larger teacher model.

1. Starting with a 15B parameter model, importance scores are computed for various components—layers, neurons, attention heads, and embedding channels. Each component is ranked according to these scores.

2. The 15B model is pruned to a smaller 8B configuration using neural architecture search.

3. The pruned model is retrained through knowledge distillation, using the original 15B model as the teacher and the 8B model as the student.

4. The 8B model then serves as the basis for a second round of pruning and distillation to produce a compact 4B model. Each model obtained in a prior stage is used as the initialization point for the next, thereby forming a sequential cascade of pruning and distillation (see Figure 4).

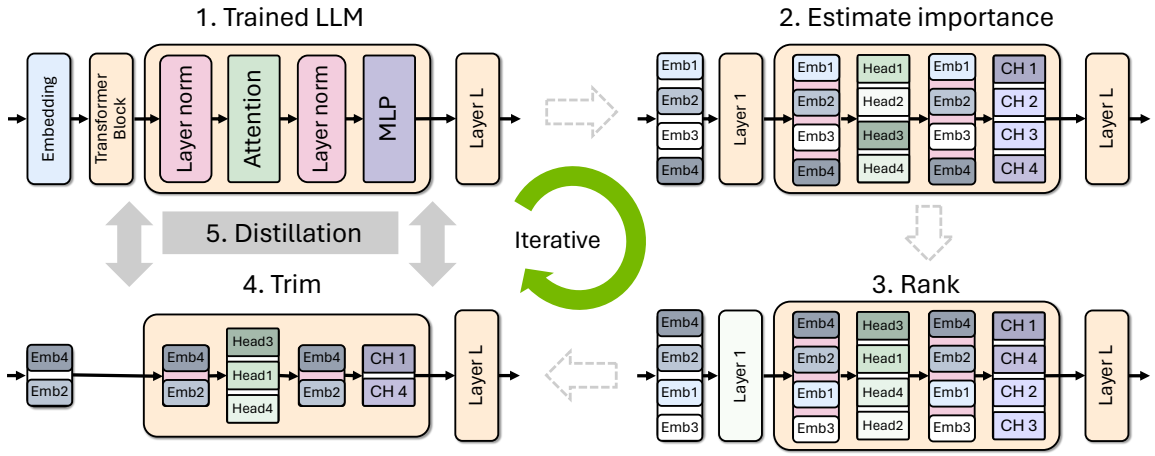



Figure 3: Overview of iterative pruning and distillation approach in Minitron. Source: Muralidharan et al. (2024)

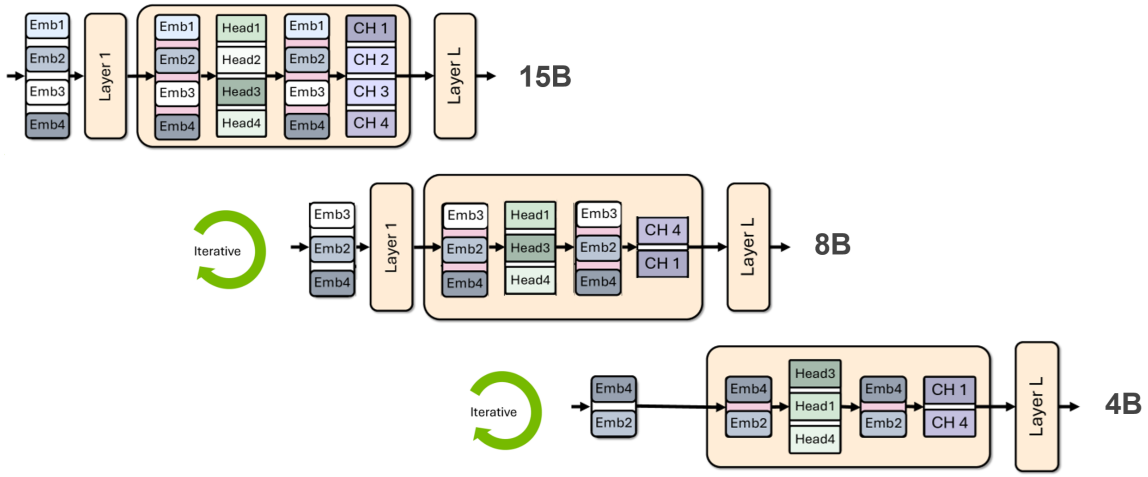



Figure 4: Illustrated example of the iterative procedure producing a family of models sequentially. Note the importance ranking may differ across iterations.

### 3.2.1 Importance Estimation

Identifying the importance or sensitivity of different network components is a foundational step in structured pruning (Muralidharan et al., 2024). Traditional metrics, such as weight magnitude, are known to be inadequate for LLMs. Instead, recent approaches favor activation-based methods and other proxies such as gradients, cosine similarity, or perplexity-based evaluations (Kim et al., 2024).

Given the prohibitive cost of computing gradients on large models, Minitron adopts a purely activation-based importance estimation strategy. This method calculates importance across all four pruning axes—depth, neuron, head, and embedding channel—using only forward passes over a small calibration dataset (1024 samples). Importance scores are derived from layer activations as follows:

**Width (Heads, Neurons, Embedding Channels):** For each axis, activations are gathered from relevant layers. The formal definitions will follow the notation outlined in Appendix C.

- **Attention Heads:** Importance is computed from the output of multi-head attention (MHA). For the $i^{\text{th}}$ head,

$$F_{\text{head}}^{(i)} = \sum_{B,S} \| \operatorname{Attn}(\mathbf{X}\boldsymbol{W}^{Q,i}, \mathbf{X}\boldsymbol{W}^{K,i}, \mathbf{X}\boldsymbol{W}^{V,i}) \|_2$$

  where $\sum_{B,S}$ refers to aggregation along the batch and sequence dimensions.

- **Neurons:** Derived from multi-layer perceptron (MLP) layer activations.

$$F_{\text{neuron}}^{(i)} = \sum_{B,S} \mathbf{X}\left(\boldsymbol{W}_1^i\right)^T$$

  where $\boldsymbol{W}_1^i$ denotes the $i^{\text{th}}$ row of the weight matrix $\boldsymbol{W}_1$.

- **Embedding Channels:** Measured from outputs of layer normalization operation (LayerNorm).

$$F_{\text{emb}}^{(i)} = \sum_{B,S} LN(\mathbf{X})_i$$

Various aggregation functions are evaluated—mean of absolute values, L2 norm, and variance—across batch and sequence dimensions to assess their effect on final model performance (see Table 3).

**Depth (Layers):** For evaluating layer importance, two complementary strategies are used.

- **Perplexity (PPL) Change:** Each layer is temporarily removed, and the change in PPL over a calibration dataset is measured (Kim et al., 2024). A small PPL change indicates redundancy. Perplexity scores are computed using:

$$\text{PPL}_n = \exp\left(-\frac{1}{SL}\sum_{s=1}^{S}\sum_{l=1}^{L}\log p_{\theta_n}(\mathbf{X}_l^{(s)} \mid \mathbf{X}_{<l}^{(s)})\right)$$

  where $\theta_n$ denotes the model with the $n^{\text{th}}$ layer removed. $\mathbf{X}_l^{(s)}$ represents the $l^{\text{th}}$ token in sequence $s$. PPL is derived from prediction loss on next token and computed with only forward pass. Early and late layers are typically preserved due to their higher influence on performance.

- **Block Importance (BI):** This approach computes the cosine distance between the input and output of a layer, defined as:

$$\mathrm{BI}_i = 1 - \mathbb{E}_{\mathbf{X},t}\left[\frac{\mathbf{X}_{i,t}^T \mathbf{X}_{i+1,t}}{\|\mathbf{X}_{i,t}\|_2 \|\mathbf{X}_{i+1,t}\|_2}\right]$$

  where $\mathbf{X}_{i,t}$ represents the $t^{\text{th}}$ token's embedding at layer $i$. More significant layers have a larger BI corresponding to more dissimilar input and output. BI is more computationally efficient, requiring only one forward pass to compute for all layers.

#### 3.2.2 Model Pruning Strategy

Once importance scores are obtained, model components across each axis are ranked and pruned using NAS-guided structural adjustments. For instance, neurons and attention heads are removed by reshaping weight matrices in MLP and MHA layers respectively, while embedding dimensions are pruned across MLP, MHA, and LayerNorm components.

An advanced technique is introduced during attention head pruning to recover information from removed heads. Inspired by the Layer Collapse method (Yang et al., 2024), residual signals from pruned heads are integrated into the remaining heads. Specifically, when pruning from $L$ to $K$ heads, each retained head $i$ is updated as:

$$\mathrm{head}_i + (\mathrm{head}_i - \mathrm{head}_{2K-i+1}), \quad i \in [K - (L-K), K]$$

For grouped-query attention (GQA) (Ainslie et al., 2023), this correction is applied only to query heads. This strategy improves performance by preserving representational diversity lost during pruning.

## 4 Neural Architecture Search

Neural architecture search (NAS) is a core component of AutoML, aiming to automate the process of neural network design to achieve optimal trade-offs between model performance and resource efficiency. Traditional deep learning has greatly benefited from automating feature extraction, but the manual design of increasingly complex architectures has become a bottleneck. NAS addresses this by algorithmically searching for high-performing architectures tailored to a given task, dataset, and resource constraint (Elsken et al., 2019). NAS has already surpassed human-designed models in vision tasks such as image classification, object detection, and semantic segmentation.

NAS operates along three main dimensions:

- **Search Space:** Defines the set of all possible architectures that is considered. Incorporating prior domain knowledge can narrow the space and make search more tractable but may also constrain novelty.

- **Search Strategy:** Determines how this space is explored—balancing the need to discover high-performing architectures quickly while avoiding premature convergence to suboptimal regions.

- **Performance Estimation Strategy:** Evaluates the effectiveness of each candidate architecture. While full training and validation is the most accurate method, it is also computationally expensive. As mentioned in Klein et al. (2024), recent research has developed faster proxy measures, such as learning curve extrapolation, early stopping, and weight sharing.

Despite its promise, NAS is computationally demanding, often requiring the training and evaluation of hundreds or thousands of candidate architectures. This challenge has spurred various efficiency improvements. Weight-sharing methods train a single *super-network* encompassing the entire search space, allowing individual *sub-networks* to share weights and drastically reduce training costs. However, the reliability of this approach is sensitive to the design of the search space and often does not outperform random sampling (Li and Talwalkar, 2020). To address this, the BigNAS approach has been introduced (Yu et al., 2020), where the super-network is trained by updating individual sub-networks, and optimal sub-networks are then selected without further retraining, reducing both computational overhead and architectural bias. The corresponding algorithm will be further described in Section 4.2.1.

Recent work also explores multi-objective NAS (see Section 4.2.2), optimizing not only for accuracy but also for latency, model size, and memory footprint on specific hardware platforms (Cai et al., 2020). Such practical constraints are increasingly relevant for deploying LLMs at scale.

### 4.1 Lightweight NAS in Minitron

In the development of the Minitron model (Muralidharan et al., 2024), NAS is used as a practical and efficient tool for structured pruning, aligning model architecture with both accuracy and parameter-efficiency goals. After estimating and ranking the importance of each component (layers, neurons, attention heads, embedding channels) through activation-based metrics (see Section 3.2.1), NAS identifies candidate sub-architectures that meet a predefined parameter budget within the search space.

Following the scheme in Table 2, the search space is constructed by enumerating feasible configurations using commonly adopted neuron, head, and embedding sizes within a narrow target parameter range (<1B). This constraint ensures the search remains tractable (typically fewer than 20 candidates). Each candidate undergoes a *lightweight retraining* phase—using around 1.8B tokens—which refines the relative performance ranking and helps stabilize the selection of optimal architectures (see Figure 13 on how retraining improves ranking consistency). This retraining step acts as an inexpensive proxy for full pretraining and significantly reduces the search cost compared to traditional NAS pipelines. Figure 5 from the original work illustrates the overall pipeline, while Figure 6 shows an illustrated example of how NAS is executed in this framework.

| Target | Layers | Heads | MLP Exp. Factor | Embedding |
|---|---|---|---|---|
| Minitron 8B | `[29-32]` | `{32,48}` | `{2.5,3,3.5,4}` | `{4096,4680,5120,5632,6144}` |
| Minitron 4B | `[29-32]` | `{24,32,48}` | `{2.5,3,3.5,4}` | `{2560,3072,3584,4096,4608}` |

Table 2: Minitron architecture search space. Source: Muralidharan et al. (2024)

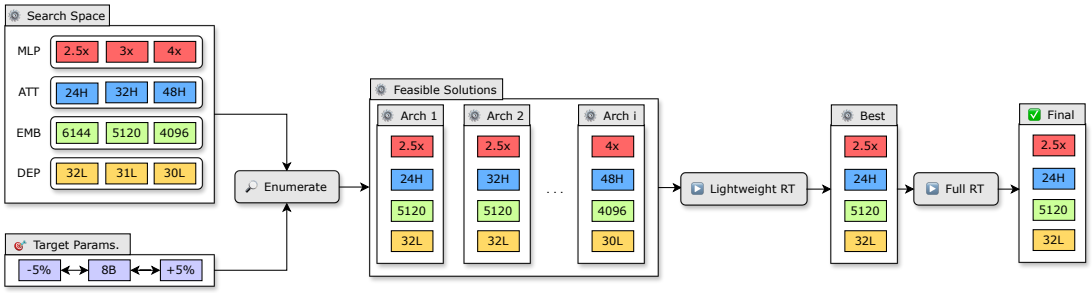



Figure 5: Overview of NAS algorithm in Minitron. RT refers to retraining. Source: Muralidharan et al. (2024)

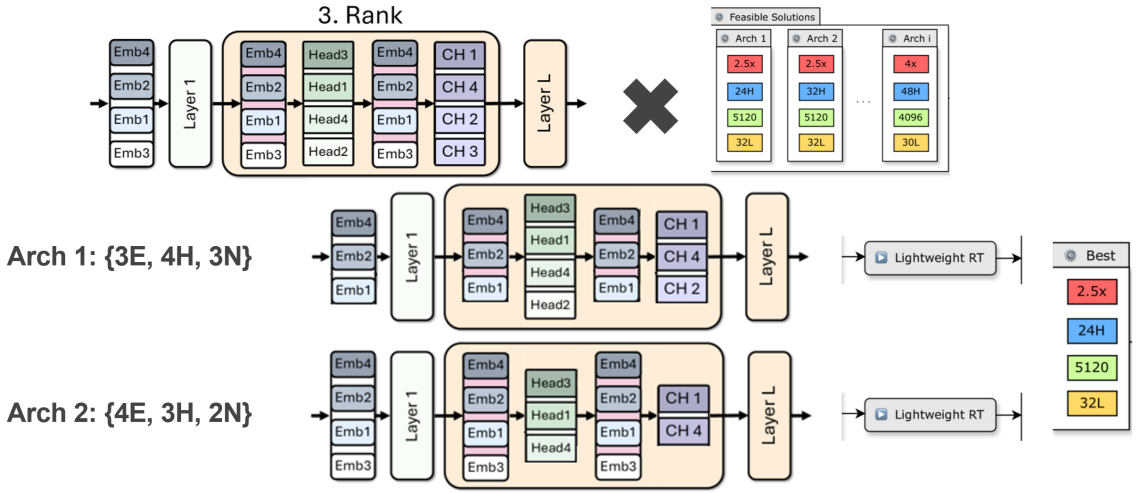



Figure 6: Example of how NAS is implemented in Minitron: After the importance ranking is obtained, one of the feasible network architecture candidate is chosen from the search space. For example, Architecture 1 here with `{3 embedding channels, 4 MHA heads and 3 MLP neurons}`. Then, the full network would be pruned in each axis by dropping the least important components until the target dimensions are reached. The process is repeated for all feasible candidates, among which best performers will be selected after lightweight retraining. (Depth pruning is omitted here for simplification. Also for demonstration purpose, MLP search space is treated as discrete number of neurons instead of multiplicative factor of embedding size.)

Although the current implementation does not integrate parameter-efficient fine-tuning techniques like LoRA (Hu et al., 2022), the framework is compatible with such enhancements.

Overall, NAS in Minitron exemplifies a pragmatic balance between AutoML principles and real-world efficiency constraints, demonstrating that structured pruning via NAS is both scalable and effective for LLM compression.

### 4.2 Pruning with NAS

NAS offers a principled and automated approach to pruning, enabling the discovery of compact, task-specific architectures that retain strong performance. Traditionally, pruning methods rely on hand-designed heuristics or fixed thresholds to reduce model capacity, often requiring extensive retraining or trial-and-error experimentation. In contrast, NAS provides a search-driven framework for pruning that can jointly optimize architecture structure—such as depth, width, or component sparsity—along with objectives like latency, model size, and accuracy (Cheng et al., 2024).

Recent works have leveraged NAS to guide structured pruning at multiple levels. For CNNs, methods like NPAS (Li et al., 2021) perform compiler-aware pruning by selecting filter types, kernel sizes, and pruning rates. Other methods for CNNs such as TAS (Dong and Yang, 2019) explore NAS to identify optimal network depth and width, training the resulting pruned networks via knowledge distillation. In the Transformer domain, NAS has been applied to discover structurally pruned architectures from fine-tuned models, offering a more data- and task-aligned alternative to conventional pruning (Klein et al., 2024).

#### 4.2.1 BigNAS

One-shot NAS has emerged as a widely adopted paradigm for efficiently ranking candidate architectures using a single set of shared weights across the search space. While this method significantly reduces the computational cost of evaluating architectures, it introduces a major limitation: the shared weights, though useful for relative ranking, often yield sub-optimal absolute performance when compared to individually trained models. As a result, standard one-shot NAS workflows typically require subsequent fine-tuning, retraining, or post-processing of the selected architectures to close the accuracy gap. These additional steps not only increase the computational burden but also complicate the overall NAS pipeline and delay model deployment.

To address the inefficiencies associated with post-processing in one-shot NAS, BigNAS (Yu et al., 2020) proposes a unified and scalable approach that eliminates the need for per-model fine-tuning. BigNAS trains a weight-shared super-network that spans a broad spectrum of sub-network architectures, ranging in size from 200 to 1000 MFLOPs. Unlike traditional approaches, BigNAS enables direct extraction and deployment of sub-networks without any further optimization, thus significantly reducing complexity. The resulting model family, referred to as BigNASModels, achieves state-of-the-art accuracy on ImageNet benchmarks and outperforms contemporary architectures such as EfficientNet and Once-for-All under similar computational constraints. This design offers both scalability and practicality by making architecture search and deployment seamless and efficient.

A critical challenge in training super-networks lies in avoiding co-adaptation among sub-networks, which can impair their standalone performance upon extraction. To mitigate this, BigNAS employs the *sandwich rule* (Yu and Huang, 2019), a training strategy that balances gradient updates across the architecture space. Specifically, at each iteration,

gradients are computed not only on the full super-network $\mathbf{w}_\Theta$ but also on $k$ randomly sampled sub-networks $\mathbf{w}_{\theta_i}$ from the search space $\Theta$:

$$\delta_t = \nabla_{\mathbf{w}} \mathcal{L}_{\text{train}}(\mathbf{w}_\Theta) + \sum_{i=1}^{k} \nabla_{\mathbf{w}} \mathcal{L}_{\text{train}}(\mathbf{w}_{\theta_i}),$$

where $\mathcal{L}_{\text{train}}$ is the training loss, typically the negative log-likelihood in language models. This gradient aggregation strategy ensures equitable training across the spectrum of model sizes, thereby improving the prunability and generalization of the sub-networks. While earlier implementations also included the smallest sub-network $\theta_{\min}$ in each update step, empirical analysis suggests it has limited impact on performance and can be omitted.

To further enhance the performance of sub-networks, BigNAS incorporates *in-place knowledge distillation*, a technique that enforces consistency between the predictions of sub-networks and the full super-network. Inspired by classical knowledge distillation (Hinton, 2014), this method minimizes the Kullback-Leibler (KL) divergence between the softmax outputs of the super-network $\pi_\Theta$ and a given sub-network $\pi_\theta$, with temperature scaling $T$:

$$\mathcal{L}_{\text{KD}} = \mathcal{L}_{\text{train}} + D_{\text{KL}} \left( \sigma_{\text{SM}} \left( \frac{\pi_\Theta}{T} \right) \| \sigma_{\text{SM}} \left( \frac{\pi_\theta}{T} \right) \right),$$

where $\sigma_{\text{SM}}(\cdot)$ denotes the softmax function. Notably, since the sandwich rule already requires forward passes through the full super-network, the teacher predictions $\pi_\Theta$ are readily available, enabling distillation to be performed in-place without incurring additional computational cost. Alternative loss metrics, such as reverse KL divergence, cosine similarity, or mean squared error, may also be used in place of the KL divergence.

Together, the sandwich rule and in-place distillation form the backbone of BigNAS training, allowing sub-networks to inherit both strong performance and generalization ability from the super-network without requiring retraining.

#### 4.2.2 Pareto Set of Sub-networks obtained by NAS

A growing body of work extends NAS for structural pruning in pre-trained language models, such as BERT and RoBERTa, to produce a diverse set of sub-networks that optimally trade off accuracy and efficiency. These NAS-based approaches adopt a multi-objective optimization framework that yields a Pareto front of candidate architectures, allowing practitioners to choose models based on application-specific constraints—such as GPU memory limits or inference latency—without re-running the search (Klein et al., 2024; Sukthanker et al., 2024).

To reduce computational cost, recent methods employ the weight-sharing approach from BigNAS, where the super-network is trained once and sub-networks are sampled or ranked using the shared weights. This setup requires only a single fine-tuning run, as sub-networks are selected and evaluated post hoc, easing the work to obtain the Pareto front. Notably, sampling strategies are refined to focus training updates on promising configurations—for instance, by reordering or re-weighting components based on learned importance scores (Sukthanker et al., 2024).

Compared to traditional threshold-based pruning, this NAS framework captures the nonlinear trade-offs between sparsity and generalization performance (see Figure 7). The Pareto-optimal set offers flexible and interpretable control over the compression process.

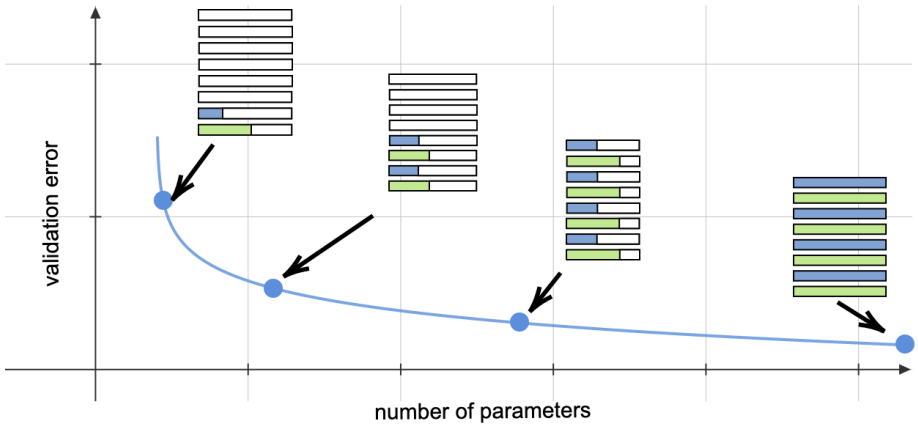



Figure 7: Multi-objective search to obtain the Pareto front of sub-networks. Horizontal bars show how binary masks are applied on the pretrained network per layer. Source: Klein et al. (2024)

As these approaches mature, future directions may include extending them to instruction-tuned or dynamic benchmarks, and incorporating distillation techniques to further boost sub-network performance.

Overall, NAS-based pruning enables a more automated, scalable, and performance-aware approach to model compression, bridging the gap between general-purpose pruning and task-optimized architectural design.

## 5 Distillation

Following the structured pruning stage, a common necessity is the restoration of model performance lost due to pruning, often referred to as *retraining* (Muralidharan et al., 2024). In this context, we consider two principal strategies:

- **Conventional Training:** The pruned model is retrained using ground truth labels to directly minimize a supervised loss, for example, cross-entropy.

- **Knowledge Distillation (KD):** Instead of relying solely on ground truth, the compact student model learns from a larger, unpruned teacher model, which provides supervisory signals through soft labels or internal representations. It facilitates the transfer of generalization behavior and predictive capacity from the teacher model to the student model, thereby preserving accuracy while reducing computational footprint, and is particularly effective in model compression (Wang et al., 2024b).

The growing dominance of LLMs, many of which are proprietary or closed-source, has led to practical constraints in performing traditional distillation, especially when internal access

to model parameters or intermediate activations is restricted. Consequently, KD approaches can be categorized based on the extent of access to the teacher model [4]:

**Black-Box Distillation (SDG Fine-Tuning):** In this scenario, the student model is fine-tuned on synthetic data generated (SDG) by querying the teacher model. The student attempts to replicate only the final predicted tokens of the teacher. This method is viable even when the teacher is inaccessible internally, for example, API-only LLMs.

**White-Box Distillation (Classical KD):** In contrast, white-box KD assumes full access to the teacher's internal representations, such as logits and hidden states. The student not only mimics the output distribution but also aligns its intermediate states with those of the teacher. This richer supervision provides more informative gradients and typically results in improved convergence and accuracy. However, this approach requires significant memory bandwidth and software framework support due to the size of intermediate outputs.

Importantly, these two approaches are not mutually exclusive; rather, they are complementary. White-box distillation yields greater fidelity when internal access is available, while black-box distillation provides a practical fallback in restrictive environments.

### 5.1 Distillation in Minitron

Minitron (Muralidharan et al., 2024) adopts a white-box distillation framework in which the uncompressed model serves as the teacher and the pruned variant as the student. The method incorporates both output-level and intermediate-level supervision to guide the retraining process. Figure 8 further illustrates the components of the distillation pipeline.

**Logit-Based Loss:** The output distribution for a given token $x_i$ is computed using a temperature-scaled softmax:

4. NVIDIA technical blog: How to Prune and Distill Llama-3.1 8B to an NVIDIA Llama-3.1-Minitron 4B Model

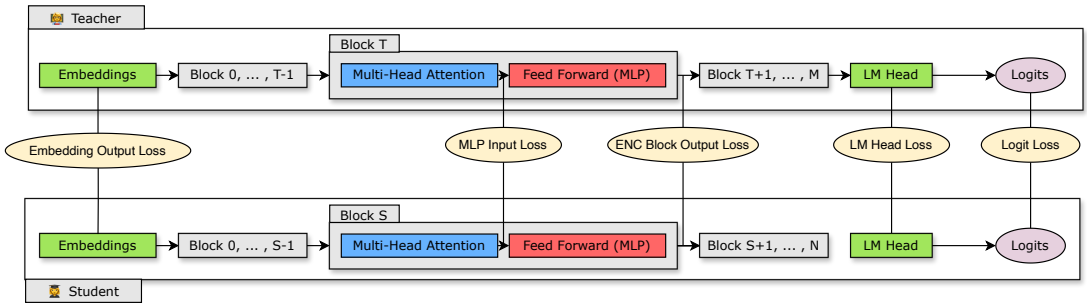



Figure 8: Overview of Knowledge Distillation in Minitron. Source: Muralidharan et al. (2024)

$$p(x_i, \tau) = \frac{\exp\left(\frac{x_i}{\tau}\right)}{\sum_{j=1}^{|V|} \exp\left(\frac{x_j}{\tau}\right)}, \tag{1}$$

where $\tau$ denotes the softmax temperature and $|V|$ is the vocabulary size. The logit-based KD loss, averaged over a sequence of $l$ tokens, is defined as:

$$L_{\text{logits}} = \frac{1}{l} \sum_{k=1}^{l} \text{Loss}(p_t^k(x, \tau), p_s^k(x, \tau)), \tag{2}$$

where $p_t^k$ and $p_s^k$ are the teacher and student output distributions as in (1) for the $k^{\text{th}}$ token, respectively.

**Intermediate State Alignment:** Beyond logits, Minitron aligns intermediate hidden states across selected Transformer layers. The intermediate state loss is given by:

$$L_{\text{is}} = \frac{1}{l} \sum_{k \in H} \sum_{i=1}^{l} \text{Loss}_k(h_t^{ki}, h_s^{ki}), \tag{3}$$

where $H$ is the set of selected hidden layers, and $h_t^{ki}$ and $h_s^{ki}$ denote the teacher and student hidden states for token $i$ at layer $k$, respectively. The hidden states are extracted post LayerNorm. To accommodate dimensional mismatches between teacher and student, a shared linear transformation is learned to project lower-dimensional student states onto the teacher's dimension. See Appendix E for a decomposition view of intermediate state loss by components.

**Total Distillation Loss:** The final objective function combines supervised learning loss and both forms of KD loss:

$$L = L_{\text{CLM}} + L_{\text{logits}} + \alpha \cdot L_{\text{is}},$$

where $L_{\text{CLM}}$ is the conventional cross-entropy loss of the student model on ground truth labels, $L_{\text{logits}}$ as in (2), $L_{\text{is}}$ as in (3), and $\alpha$ is a weighting coefficient. Given the differing magnitudes of $L_{\text{logits}}$ and $L_{\text{is}}$, dynamic scaling of $\alpha = \frac{L_{\text{logits}}}{L_{\text{is}}}$ yielded more stable training dynamics and improved performance relative to using a fixed value.

### 5.2 NAS and Distillation

While increasing LLMs model size typically yields higher accuracy, it also exacerbates the discrepancy between state-of-the-art performance and real-world deployability. To address this, a hardware-aware NAS and distillation framework *Puzzle* (Bercovich et al., 2025) is designed to optimize LLMs for efficient inference without compromising their capabilities.

#### 5.2.1 Puzzle

LLMs have traditionally adopted highly uniform architectures, comprising repeated identical layers, primarily to ensure training stability and facilitate scaling. While this homogeneity benefits pretraining, it fails to account for the varying computational contributions of

individual layers to overall predictive performance, leading to inefficiencies in inference. In deployment scenarios—especially on specific hardware such as NVIDIA H100 GPUs—these inefficiencies manifest as elevated latency, suboptimal throughput, and excessive memory consumption. The central challenge, therefore, lies in transforming a well-trained LLM from a training-optimized architecture into an inference-optimized architecture, while preserving its accumulated knowledge and predictive capabilities.

Bercovich et al. (2025) introduce Puzzle, a hardware-aware framework that systematically explores a vast space of heterogeneous model configurations to produce inference-efficient variants of a given parent model. Puzzle accommodates multiple deployment constraints, including latency, throughput, and memory limits, while scaling to tens of billions of parameters with only a fraction of the original training compute.

Puzzle introduces a *decomposed NAS* strategy inspired by prior work in computer vision, for example, LANA (Molchanov et al., 2022), but adapted to the unique scale and structure of LLMs. As illustrated in Figure 9, the framework decomposes the search problem into modular stages:

1. **Block Library Construction:** A library of block variants is constructed using *blockwise local distillation* (BLD), enabling efficient parallel training and evaluation of block replacements.

2. **Resource Estimation:** Each candidate block is profiled for memory and latency across various input lengths and hardware targets.

3. **Block Scoring:** Blocks are scored based on accuracy preservation and efficiency tradeoffs when substituted in specific network locations.

4. **Architecture Search:** A constrained *mixed-integer programming* (MIP) solver identifies full model architectures composed of block candidates that jointly maximize performance while satisfying deployment constraints.

5. **Global Distillation:** The final stage is *global knowledge distillation* (GKD), which performs short, end-to-end finetuning on the assembled architecture to align all blocks jointly.

The design space is comprised of alternative blocks in the parent LLM, each of which are composed of smaller subblocks: attention module and feed-forward network (FFN). The subblocks have options of varying computational complexity, including the extreme case of no-op (skipped) layers. To efficiently navigate this large search space, Puzzle introduces BLD to train candidate blocks independently and in parallel. Each block is distilled to mimic the function of its corresponding parent block, relying solely on parent activations. This decomposition offers three key benefits: (i) computational isolation across blocks enables pipeline-parallel training on multiple GPUs; (ii) each block learns a comparatively simple mapping, yielding faster convergence and allowing higher learning rates; and (iii) the process requires only a small dataset (approximately one billion tokens) while benefiting from high-quality upstream activations.

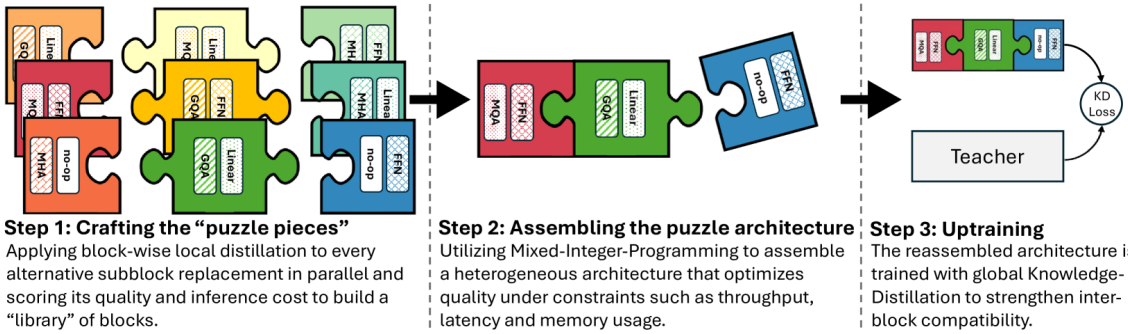



Figure 9: An overview of the Puzzle decomposed NAS framework. Source: Bercovich et al. (2025)

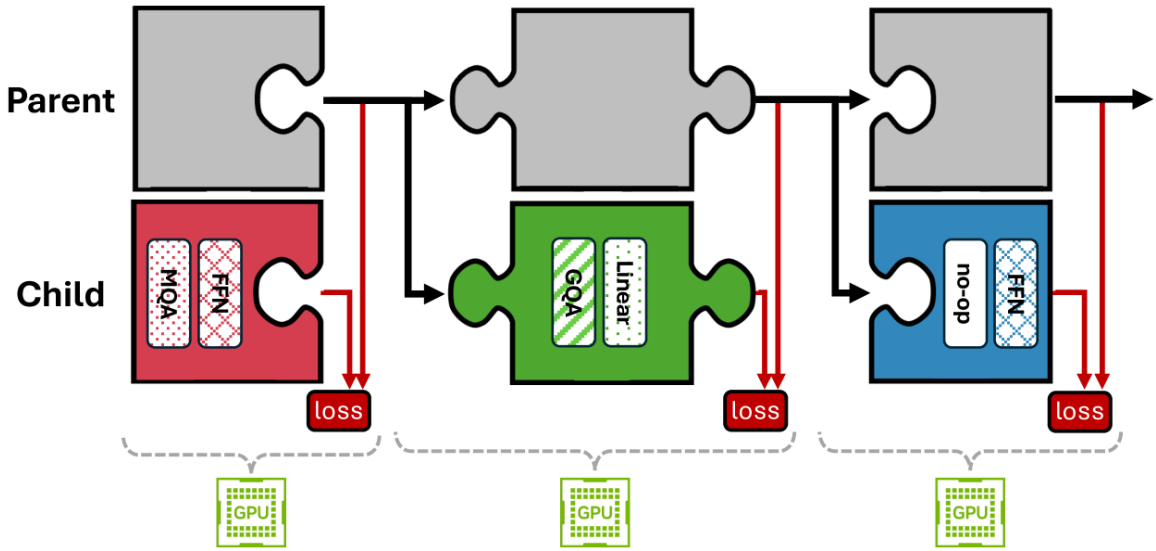



Figure 10: BLD is a hardware-aware distillation approach that train in parallel and independently for each block. Source: Bercovich et al. (2025)

**Decoupled BLD:** A variant of traditional blockwise distillation that separates the optimization of each subblock (attention or FFN) from others. This significantly increases search throughput and scalability, allowing exploration of a vastly larger design space within a single optimization cycle. Decoupled BLD also paves the way for integrating novel architectural primitives such as variable-window attention, state-space models, and other latency-optimized operations. Notably, the decoupled BLD formulation reduces block library training cost from $m \cdot n \cdot l$ variants to $(m+n) \cdot l$ variants for $m$ attention types, $n$ FFN types, and $l$ transformer layers, transforming the search space complexity from multiplicative to additive.

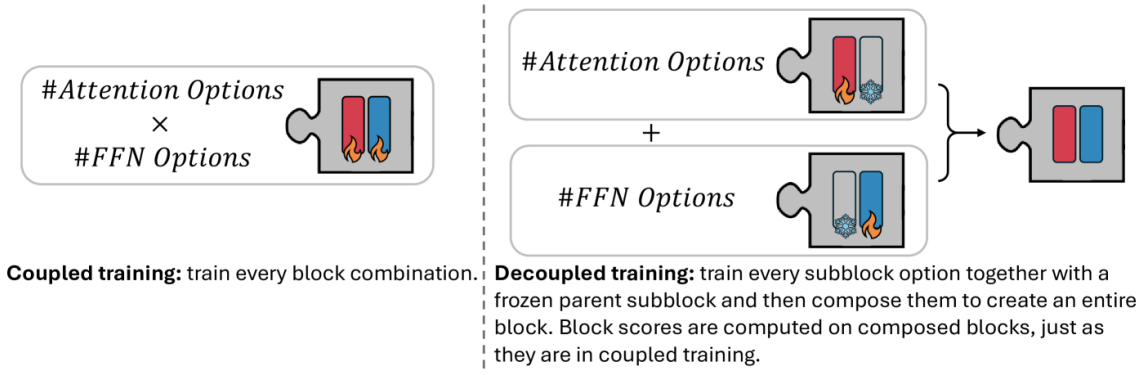



Figure 11: Decoupled BLD requires training variants per layer of additive-complexity only, significantly speeding up library construction. Source: Bercovich et al. (2025)

The Puzzle search space is combinatorially large: given the number of possible attention and FFN variants per layer, evaluating even a representative subset of full models is computationally infeasible. To overcome this, Puzzle scores each candidate block independently using a *replace-one-block* evaluation. In this scheme, the parent model is modified to replace a single block with a trained variant, and the resulting performance is measured. Scores are cached for reuse during search, ensuring that each block is evaluated only once. This enables rapid estimation of any assembled architecture's quality as the sum of its constituent block scores.

The architecture selection process is formulated as a grouped variant of the *Knapsack Problem*, where each layer forms a group containing multiple block alternatives. Each alternative has a quality score and a multi-dimensional cost profile (parameter memory, KV-cache memory, and runtime). A MIP solver is employed to select exactly one block per layer, maximizing total quality while satisfying deployment constraints. Importantly, this MIP-based selection operates immediately after the BLD stage, avoiding the need for full-model distillation for each candidate. Only the final, selected architectures for each deployment constraint tier proceed to the GKD (Hinton, 2014) stage, significantly reducing computational cost.

A limitation of BLD is the absence of inter-block compatibility tuning: each block is trained with parent outputs rather than the outputs of preceding child blocks, potentially leading to compounding errors. This is addressed in the final stage through GKD, resulting in a non-uniform Transformer with adaptive computational allocation across layers. This stands in contrast to traditional uniformly structured models and enables better utilization of available compute resources per layer.

A central benefit of decomposed NAS lies in its capacity to approximate the performance of a candidate model by aggregating metrics derived from its constituent blocks. This decomposition enables highly efficient traversal of vast architectural search spaces: rather than constructing full models and evaluating their performance through costly forward passes (for example, validation accuracy), candidate architectures can be evaluated in under a

second using precomputed scores. In contrast, conventional NAS techniques—particularly those employed in computer vision—typically require full model training or evaluation, which is computationally infeasible in the context of LLMs due to their expansive architectures and inference costs.

Empirical results validate Puzzle's effectiveness. For example, Puzzle-optimized models: Llama-3.1-Nemotron-51B-Instruct and Llama-3.3-Nemotron-49B, both achieve up to 2.17× throughput improvement on a single NVIDIA H100 GPU, while retaining up to 98.4% of the original LLaMA-70B's accuracy. Notably, these models are trained with just 45B tokens, compared to the 15T required for the original, demonstrating exceptional data and compute efficiency. Moreover, lightweight post-distillation alignment enables these derived models to surpass their parent model in specialized tasks, such as instruction following or retrieval-augmented generation. While methods such as Minitron (Muralidharan et al., 2024), ShortGPT, and SlimGPT share conceptual similarities with Puzzle, they operate over narrower optimization spaces. For instance, Minitron enforces homogeneous block replacements across all layers, whereas Puzzle supports heterogeneous, layer-specific configurations, diverse block types (including skips), and global pruning ratios optimized jointly via MIP. This flexibility, coupled with decomposed evaluation and staged distillation, enables Puzzle to efficiently generate multiple hardware-specialized child models from a single high-performance parent, pushing the accuracy-per-dollar frontier.

#### 5.2.2 MiniPuzzle

Building on Puzzle and Minitron advancements, *MiniPuzzle* (Blakeman et al., 2025) is introduced as a simplified yet effective framework that combines structured pruning with conditional NAS and lightweight distillation. Designed to meet deployment-specific constraints (for example, 32GiB GPU memory), MiniPuzzle follows a three-stage pipeline:

1. **Importance Estimation:** Adopting the block scoring technique from Puzzle, each layer's contribution is scored via a proxy metric: the mean squared error (MSE) between the intermediate activations of the full model and those of a pruned variant over a small input sample (128 examples). This yields per-layer importance scores. FFN importance is calculated in the same manner as Minitron.

2. **Conditional NAS:** Using these scores, MiniPuzzle explores reduced architectures that preserve the most important components while satisfying memory and latency constraints.

3. **Distillation:** The selected architecture is distilled from the full model using a lightweight training regime to recover accuracy lost during pruning.

Figure 12 outlines this process, highlighting the efficient combination of pruning, distillation and NAS for producing deployment-ready models, which is employed by NVIDIA to obtain the Nemotron-H model family.

The findings demonstrate that NAS and distillation, when tightly coupled in a hardware-aware framework, offer a powerful approach to LLM compression. Puzzle and MiniPuzzle jointly establish a new frontier in efficient LLM deployment, particularly for resource-constrained settings such as on-device inference or low-latency APIs. Ultimately, this line of

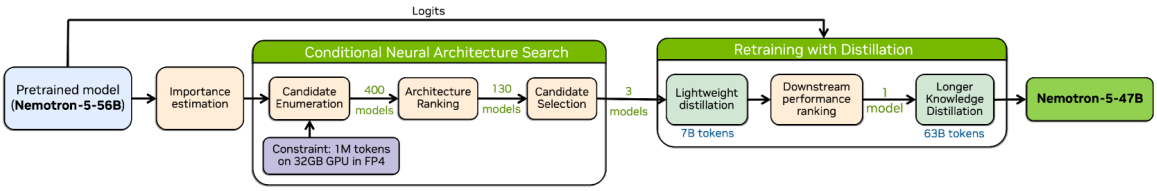



Figure 12: MiniPuzzle's optimization process: a) estimate layer and FFN importance scores, b) search for the best model candidates using the conditional NAS framework, c) select the best candidate with lightweight distillation and recover the accuracy lost with longer distillation. Source: Blakeman et al. (2025)

work reinforces the notion that parameter count should not be the sole metric guiding model selection. Instead, architecture-level optimization grounded in deployment constraints offers a principled pathway to scalable and cost-effective LLM deployment.

## 6 Experiments

Muralidharan et al. (2024) empirically evaluated the structured pruning and distillation approach on the Nemotron-Minitron family. Specifically, the Nemotron-4 model with 15.6B parameters is compressed into two smaller Minitron variants targeting approximately 8B and 4B parameters, respectively. All pruning and retraining procedures are implemented using the NVIDIA Megatron-LM framework.

**Data and Training Configuration:** The curated 8 trillion token (8T) base pretraining dataset and the continued training (CT) dataset released by Parmar et al. (2024) are adopted. Appendix F provides a brief overview and description of the dataset.

- All ablation studies are conducted on 18.9B tokens of the 8T pretraining blend.
- For importance estimation, a calibration dataset $D$ comprising 1024 randomly sampled examples from the full corpus is used.
- Unless otherwise specified, lightweight retraining is performed over 1.8 billion tokens (corresponding to 400 steps).
- Final retrained models are obtained using 94B tokens of a combination of both the 8T and CT datasets.

Optimization hyperparameters follow Parmar et al. (2024), including a cosine learning rate schedule that decays from $2^{-4}$ to $4.5^{-7}$.

**Evaluation Benchmarks:** Following Llama-2 (Touvron et al., 2023), the pruned and retrained models are evaluated on a broad suite of downstream tasks covering reasoning, code generation, and summarization. Tasks include: MMLU (Hendrycks et al., 2021), HumanEval

(Python code generation), ARC-Challenge, HellaSwag, TruthfulQA, WinoGrande, and XL-Sum (English). 5-shot results for MMLU and WinoGrande, 25-shot for ARC-Challenge, 10-shot for HellaSwag, and zero-shot performance on a 20% held-out subset of XL-Sum are reported. HumanEval and MBPP scores are reported as average pass@1, computed using temperature sampling ($T = 0.2$) and nucleus sampling with top-$p = 0.95$.

Instruction-tuned variants are additionally evaluated on MT-Bench, Instruction-Following Eval (IFEval), ChatRAG-Bench, and the Berkeley Function Calling Leaderboard (BFCL).

### 6.1 Empirical Best Practices

Based on extensive ablation studies and experimental results, Muralidharan et al. (2024) summarized key takeaways in the form of structured compression best practices for structured pruning and retraining of LLMs:

1. Begin by training the largest model in the family; obtain smaller variants via iterative prune-and-distill steps.

2. Use `(batch=L2, seq=mean)` aggregation functions to estimate importance scores for pruning width axes, and PPL or BI metrics for depth.

3. Prefer single-pass importance estimation; iterative schemes (see Appendix G) show negligible benefits.

4. Width pruning is preferable over depth pruning for models up to 15B parameters.

5. Favor distillation-based retraining by KL divergence loss over conventional training.

6. For significant depth reductions, combine distillation losses from logits, intermediate states, and embeddings.

7. For moderate depth reductions, logit-only distillation is typically sufficient.

8. Always prune from a model that is closest in size to the target.

9. Perform lightweight retraining to stabilize post-pruning searched-candidate rankings.

10. For multi-phase training pipelines, prune the model produced at the final phase rather than earlier checkpoints.

Each recommendation is supported by experimental evidence. These best practices were used to derive the Minitron models, whose performance is reported in Tables 4 and 5. While these practices are empirically validated within experimental setup, we caution that their effectiveness may vary across different datasets, tasks, or model families, and they should be interpreted as guiding heuristics rather than universal rules.

### 6.2 Experiment Results

The empirical results are reported across three key settings: comparison to existing baselines, comparison to prior pruning-based approaches, and performance after instruction tuning. All evaluations are conducted on the downstream tasks described earlier in this section. Performance metrics, parameter counts (both full and non-embedding), and number of training tokens are summarized in Tables 4 through 10.

**Result 1: Comparison with Baseline and Community Models.** The pruned model is evaluated against several baselines: (1) the original Nemotron-4 15B model, (2) the earlier Nemotron-3 8B model, and (3) a set of comparably sized open-source community models trained from scratch with trillions of tokens. Notably, the Minitron models are trained using only 94 billion tokens, highlighting the efficiency of its pruning and distillation pipeline.

From Table 4, the Minitron 8B model achieves competitive or superior performance relative to popular community models such as LLaMA-2 7B, Mistral 7B, Gemma 7B, and LLaMA-3 8B. It also consistently outperforms Nemotron-3 8B—despite being trained with dramatically fewer tokens.

As shown in Table 5, the Minitron 4B model preserves general capabilities more effectively than smaller specialized models, which often exhibit performance peaks on narrow task subsets. In particular, it outperforms the Gemma-2B model across multiple tasks, demonstrating robust all-around utility.

**Result 2: Comparison with Pruned Model Baselines.** Minitron is compared to state-of-the-art structured pruning baselines, including LLM-Pruner, SliceGPT, LaCo, ShortGPT, and Sheared LLaMA (Xia et al., 2024). Architectural specifications for the Minitron models used in this comparison are detailed in Table 11, while performance results are shown in Table 6.

The Minitron 8B model substantially outperforms multiple depth-pruned baselines that have larger parameter counts (approximately 10B). Meanwhile, the Minitron 4B variant exhibits significant improvements over all listed pruning-based models in Table 6, validating the strength of its method for structured compression.

**Result 3: Instruction Tuning Performance.** To assess performance under supervised fine-tuning, instruction tuning is applied to the Minitron 4B model using the instruction dataset employed for Nemotron-4 340B (Adler et al., 2024), resulting in the Minitron 4B-instruct variant.

Evaluation results across instruction-following (IFEval, MT-Bench), retrieval-augmented generation (ChatRAG-Bench), and function-calling (BFCL) tasks are presented in Tables 7–10. The Minitron 4B-instruct model exhibits strong instruction-following and conversational abilities, outperforming similarly sized models across all tasks. Of particular note, it surpasses both Gemma-2B-IT and LLaMA-3 8B-instruct on the function-calling benchmark.

The results in Tables 4 through 10 provide strong empirical support for the core hypothesis: training a single large model and subsequently deriving smaller models via structured pruning and lightweight distillation yields superior accuracy and efficiency compared to training each model from scratch. Furthermore, the retraining protocol substantially reduces data requirements, eliminating the need to curate trillions of tokens, while achieving competitive or state-of-the-art performance across a range of evaluation settings.

## 7 Conclusion

In addition to advancing LLM compression methodology, the Minitron study contributes to the broader field in three ways: (1) it demonstrates the effectiveness of model compression for scalable LLM deployment, especially pruning and distillation; (2) it illustrates the value of NAS in identifying optimized architectures under deployment constraints; and (3) it introduces the Minitron family of SLMs, derived from larger models through principled pruning and distillation, which shows competitive performance amid significantly cheaper cost.

**Limitations:** While the experiment results are promising, they are currently limited to the Nemotron family of models. Extending the techniques to other architectures remains an important direction for future work. Additionally, although the retraining procedure is lightweight in data usage, it still requires full model retraining. Due to high computational cost of LLMs, it is impractical to report confidence intervals or statistical significance over multiple runs.

**Future Directions:** Promising avenues for further research include incorporating parameter-efficient fine-tuning strategies to further reduce retraining cost, exploring broader integration with various quantization methods, and applying advanced NAS techniques to co-optimize architecture and pruning jointly. These directions may lead to even more efficient and generalizable compression pipelines for LLM deployment.

## Appendix A. Minitron Training Cost Savings

Muralidharan et al. (2024) calculate the floating point operations (FLOPs) per training step for the 15B, 8B, and 4B models as $4.4 \times 10^{17}$, $2.5 \times 10^{17}$, and $1.2 \times 10^{17}$, respectively.[5] Training each model from scratch with the same token count, number of steps, and batch size results in a total cost of:

$$\left(4.4 \times 10^{17} + 2.5 \times 10^{17} + 1.2 \times 10^{17}\right) \times \text{steps} \tag{4}$$

However, by leveraging the pruning and distillation pipeline, only the 15B model is trained from scratch. The 8B and 4B variants are derived using lightweight distillation with 40× fewer tokens, resulting in a new total cost:

$$\left(4.4 \times 10^{17} + \frac{2.5 \times 10^{17}}{40} + \frac{1.2 \times 10^{17}}{40}\right) \times \text{steps} \tag{5}$$

This yields an approximate 1.8× reduction in end-to-end training FLOPs for the model family compared to (4).

To estimate the total computational cost (in FLOPs) for training an entire model family, we compute the product of the family-level FLOPs required per training step (from (4) or (5)) and the total number of training steps.

$$\text{FLOPs}_{\text{family}} = \text{family FLOPs per step} \times \text{steps} \tag{6}$$

The number of training steps is derived from the 15B parameter model trained on 8 trillion tokens. Specifically, we divide the total model FLOPs required to process the full training corpus by the FLOPs consumed per training step. In our case:

$$\text{steps} = \frac{\text{FLOPs}_{\text{15B-model}}}{4.4 \times 10^{17}}$$

Following the approximation introduced by Chowdhery et al. (2023), the total number of FLOPs required to train a decoder-only language model over a corpus is estimated as:

$$\text{Total Model FLOPs} = 6 \times \text{Model Parameters} \times \text{Number of Tokens} \tag{7}$$

Applying (7) to our setup:

$$\text{FLOPs}_{\text{15B-model}} = 6 \times 15.6 \times 10^9 \times 8 \times 10^{12}$$

To estimate wall-clock training time for whole family, we divide the total required FLOPs from (6) by the product of the model FLOP utilization (MFU) rate and the sustained hardware throughput (FLOPs/sec).

The total training cost (in USD) is then computed as:

$$\text{Cost} = \left(\frac{\text{FLOPs}_{\text{family}}}{\text{MFU} \times \text{throughput}}\right) \times \text{GPU hourly cost}$$

5. Assuming a batch size of 1152.

In this study, we assume a GPU hourly cost of approximately USD 2 (as of 2025 estimates). According to Parmar et al. (2024), MFU is assumed to be 30.5%, and throughput for the NVIDIA H100 GPU is 989 teraFLOP/s. For further details on FLOPs estimation and methodology, refer to the following resources:

- FLOPs Demystified: Training Cost Breakdown
- How to Calculate FLOPs in Transformer Models

## Appendix B. MMLU Score

**Massive Multitask Language Understanding (MMLU):** The benchmark is a widely adopted evaluation suite for assessing the generalization and reasoning capabilities of large language models. It measures a model's accuracy across 57 diverse tasks, spanning domains such as elementary mathematics, U.S. history, computer science, and law (Hendrycks et al., 2021).

To achieve strong performance on MMLU, models must demonstrate both broad factual knowledge and sophisticated problem-solving abilities, making it a robust test of multitask and multi-domain understanding.

## Appendix C. Notation for Minitron Importance Estimation

Muralidharan et al. (2024) formally defined three core architectural components of Transformer models as follows:

**Multi-Layer Perceptron (MLP):** An MLP layer consists of two linear transformations separated by a non-linear activation:

$$\text{MLP}(\mathbf{X}) = \delta\left(\mathbf{X} \cdot \boldsymbol{W}_1^T\right) \cdot \boldsymbol{W}_2$$

where $\mathbf{X}$ is the input, and $\boldsymbol{W}_1, \boldsymbol{W}_2 \in \mathbb{R}^{d_{\text{hidden}} \times d_{\text{model}}}$ are learnable weights, $d_{model}$ and $d_{hidden}$ are the embedding and MLP hidden dimensions, respectively. $\delta(\cdot)$ denotes the activation function.

**Multi-Head Attention (MHA):**

$$\text{MHA}(\mathbf{X}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_L) \cdot \boldsymbol{W}^O$$

$$\text{head}_i = \text{Attn}(\mathbf{X}\boldsymbol{W}^{Q,i}, \mathbf{X}\boldsymbol{W}^{K,i}, \mathbf{X}\boldsymbol{W}^{V,i})$$

Here, $\boldsymbol{W}^{Q,i}, \boldsymbol{W}^{K,i}, \boldsymbol{W}^{V,i} \in \mathbb{R}^{d_{\text{head}} \times d_{\text{model}}}$ are projection matrices for the $i$-th head, and $\boldsymbol{W}^O \in \mathbb{R}^{Ld_{\text{head}} \times d_{\text{model}}}$ projects concatenated heads back to the model dimension, where $d_{head}$ is the size of a single attention head, and $L$ is the total number of heads.

**Layer Normalization (LayerNorm):**

$$\text{LN}(\mathbf{X}) = \frac{\mathbf{X} - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta$$

where $\mu$ and $\sigma^2$ are the mean and variance across the feature dimension, $\epsilon$ is a stability constant, and $\gamma, \beta$ are learnable scale and shift parameters.

## Appendix D. Lightweight Retraining in Minitron

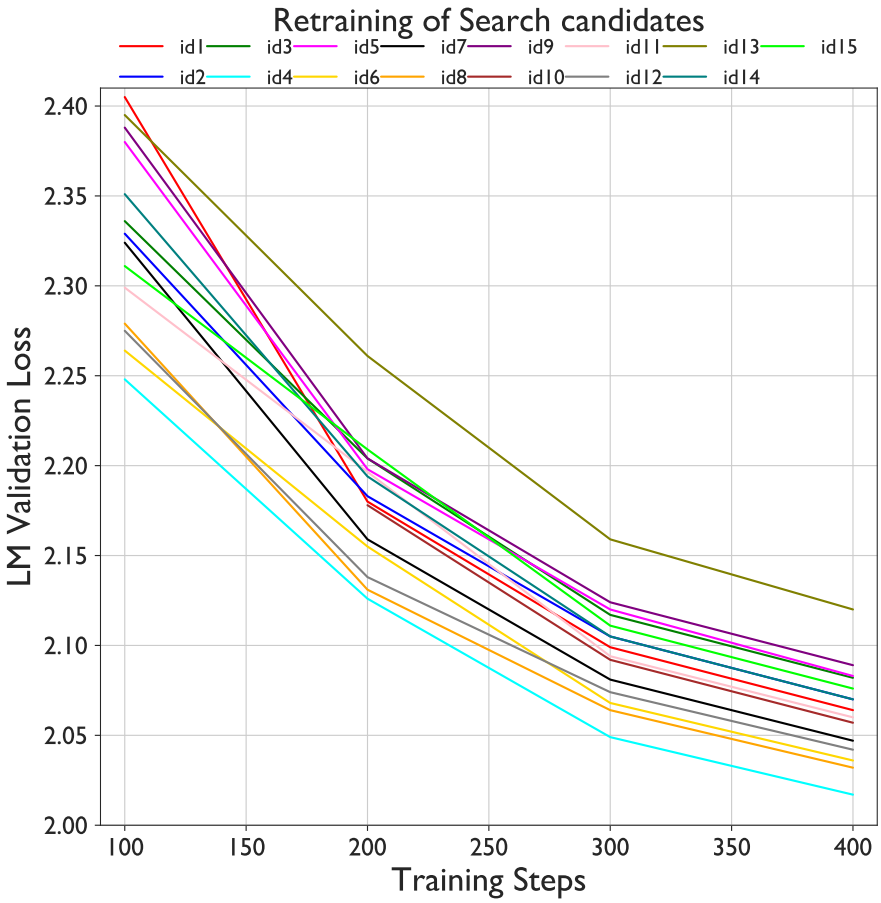



Figure 13: Lightweight retraining of searched candidates with 1.8B training tokens to stabilize the performance ranking. Source: Muralidharan et al. (2024)

## Appendix E. Minitron Intermediate State Loss

The intermediate-state-based loss $L_{\text{is}}$ for Transformer-based models is composed of multiple components, each targeting a distinct aspect of the network:

$$L_{\text{is}} = L_{\text{emb}} + L_{\text{att}} + L_{\text{i}} + L_{\text{o}}$$

- $L_{\text{emb}}$: Loss based on word embeddings.
- $L_{\text{att}}$: Attention relation loss, computed using the query, key, and value projections.
- $L_{\text{i}}$: Loss associated with the input to the MLP layers.
- $L_{\text{o}}$: Loss computed from the output activations of the Transformer blocks.

Each term contributes to guiding the learning of internal representations and improving architectural robustness.

## Appendix F. Minitron Training Datasets

According to Parmar et al. (2024), the pretraining dataset for the Minitron-8B Base model is composed of a broad mixture of English, multilingual text, and source code. The corpus encompasses diverse document types, including web pages, dialogue transcripts, scientific articles, and other written content.

Key content domains include law, mathematics, natural sciences, and finance. To enhance downstream capabilities, the continued training phase incorporates a small amount of data focused on question answering and alignment-oriented interactions. This strategy promotes better generalization and improved instruction-following behavior.

## Appendix G. Iterative Importance Estimation

A structured pruning technique in which model components (for example, layers, heads, neurons) are iteratively ranked by importance and pruned across multiple stages (Muralidharan et al., 2024).

Given a source dimension $d_s$, target dimension $d_t$, and total number of iterations $T$, the pruning schedule operates as follows: at iteration $i \in [0, T-1]$, the importance is estimated on $d_s - i \cdot \left(\frac{d_s - d_t}{T}\right)$ components and the model is pruned down to $d_s - (i+1) \cdot \left(\frac{d_s - d_t}{T}\right)$ components.

## Appendix H. Additional Tables

### H.1 Aggregation Functions for Width Importance in Minitron

| Batch | Sequence | 8T LM Loss | WikiText2 LM Loss |
|---|---|---|---|
| L2 | L2 | 8.73 | 8.37 |
| **L2** | **mean** | **7.18** | **7.23** |
| L2 | var | 8.18 | 8.61 |
| mean | L2 | 8.41 | 7.84 |
| **mean** | **mean** | **7.21** | **6.89** |
| mean | var | 7.94 | 8.29 |
| var | L2 | 9.01 | 9.30 |
| var | mean | 8.34 | 8.72 |
| var | var | 10.55 | 11.14 |

Table 3: Post-pruning (zero-shot) performance of activation-based importance with different batch and sequence aggregation metrics for width pruning, evaluated on the validation set of the 8T and WikiText2 datasets. Source: Muralidharan et al. (2024)

### H.2 Minitron Experiment Results

| | | | Models | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | **Benchmark** | **Metric** | **Llama-3** | **Llama-2** | **Mistral** | **Gemma** | **Nemotron-4** | **Nemotron-3** | **Minitron** |
| | # Parameters | | 8B | 6.7B | 7.3B | 8.5B | 15.6B | 8.5B | 8.3B |
| | # Non-Emb. Params | | 5.9B | 6.4B | 7B | 7.7B | 12.5B | 6.4B | 6.2B |
| | # Training Tokens | | >15T | 2T | 8T | 6T | 8T | 3.8T | **94B** |
| Knowledge,Logic | winogrande (5) | acc | 77.6 | 74 | 78.5 | 78 | 83.6 | 75.9 | **79.0** |
| | arc_challenge (25) | acc_norm | 57.8 | 53 | 60.3 | **61** | 58.8 | 52.8 | 52.6 |
| | MMLU(5) | acc | **65.3** | 46 | 64.1 | 64 | 66.6 | 54.7 | 63.8 |
| | hellaswag(10) | acc_norm | 82.1 | 79 | **83.2** | 82 | 84.6 | 78.5 | 80.7 |
| | gsm8k(5) | acc | 50.3 | 14 | 37 | 50 | 48.5 | 24.0 | **51.3** |
| | truthfulqa(0) | mc2 | 43.9 | 39 | 42.6 | **45** | 40.7 | 36.5 | 42.6 |
| | XLSum en (20)(3) | rougeL | 30.9 | 31 | 4.80 | 17 | 32 | 30.9 | **31.2** |
| Coding | MBPP(0) | pass@1 | **42.4** | 20 | 38.8 | 39 | 38 | 27.04 | 35.2 |
| | humaneval (n=20)(0) | pass@1 | 28.1 | 12 | 28.7 | **32** | 35.4 | 20.7 | 31.6 |

Table 4: Performance of Minitron 8B model compared to multiple baselines: the original Nemotron-4 15B, the previous generation Nemotron-3 8B, and multiple community models. Source: Muralidharan et al. (2024)

| | | | Models | | | | | |
|---|---|---|---|---|---|---|---|---|
| | **Benchmark** | **Metric** | **Phi-2** | **Gemma** | **Gemma2*** | **Qwen2*** | **MiniCPM*** | **Minitron** |
| # Parameters | | | 2.7B | 2.5B | 2.6B | 1.5B | 2.7B | 4.2B |
| # Non-Emb. Params | | | 2.5B | 2B | 2B | 1.3B | 2.4B | 2.6B |
| # Training Tokens | | | 1.4T | 3T | 2T | 7T | 1.1T | **94B** |
| Knowledge, Logic | winogrande (5) | acc | **74** | 67 | 70.9 | 66.2 | - | **74.0** |
| | arc_challenge (25) | acc_norm | **61** | 48 | 55.4 | 43.9 | - | 50.9 |
| | MMLU(5) | acc | 57.5 | 42 | 51.3 | 56.5 | 53.5 | **58.6** |
| | hellaswag(10) | acc_norm | **75.2** | 72 | 73.0 | 66.6 | 68.3 | 75.0 |
| | gsm8k(5) | acc | 55 | 18 | 23.9 | **58.5** | 53.8 | 24.1 |
| | truthfulqa(0) | mc2 | 44 | 33 | - | **45.9** | - | 42.9 |
| | XLSum en (20)(3) | rougeL | 1 | 11 | - | - | - | **29.5** |
| Coding | MBPP(0) | pass@1 | **47** | 29 | 29.6 | 37.4 | - | 28.2 |
| | humaneval (n=20)(0) | pass@1 | **50** | 24 | 17.7 | 31.1 | - | 23.3 |

Table 5: Performance of Minitron 4B model compared to similarly-sized community models. Entries marked with * are quoted from the corresponding original papers. Source: Muralidharan et al. (2024)

| | | | Models | | | | | |
|---|---|---|---|---|---|---|---|---|
| | **Benchmark** | **Metric** | **LLMPruner** | **SliceGPT** | **LaCo** | **ShortGPT** | **Sheared LLaMa** | **Minitron** |
| **8 Billion** | # Parameters | | 9.8B | 9.9B | 9.8B | 9.8B | - | 8.3B |
| | # Non-Emb. Params | | 9.5B | 9.5B | 9.5B | 9.5B | - | 6.2B |
| | MMLU(5) | acc | 25.2 | 37.1 | 45.9 | 54.7 | - | **63.8** |
| | hellaswag(10) | acc_norm | 67.8 | 55.7 | 64.4 | 66.6 | - | **80.7** |
| **4 Billion** | # Parameters | | 4.8B | 4.9B | 4.9B | 4.9B | 2.7B | 4.2B |
| | # Non-Emb. Params | | 4.5B | 4.6B | 4.6B | 4.6B | 2.5B | 2.6B |
| | winogrande (5) | acc | - | - | - | - | 64.2 | **74** |
| | arc_challenge (25) | acc_norm | - | - | - | - | 41.2 | **50.9** |
| | MMLU(5) | acc | 23.33 | 28.92 | 26.45 | 43.96 | 26.4 | **58.6** |
| | hellaswag(10) | acc_norm | 56.46 | 50.27 | 55.69 | 53.02 | 70.8 | **75** |
| | gsm8k(5) | acc | - | - | - | - | 23.96 | **24.1** |

Table 6: Performance of Minitron models compared to state-of-the-art pruned models. Source: Muralidharan et al. (2024)

| Model | Non-Emb. Params | Tokens | Total |
|---|---|---|---|
| Minitron 4B-instruct | 2.6B | **90B** | **6.46** |
| Phi-2 | 2.5B | 1.4T | 4.29 |
| Qwen-1.5 Chat | 1.2B | N/A | 5.29 |
| Gemma-2B-IT | 2B | 6T | 5.19 |
| StableLM 2 Chat | 1.6B | 2T | 5.42 |
| TinyLlama v1.0 Chat | 1.1B | 3T | 3.46 |

Table 7: Evaluation results on MT-Bench. Source: Muralidharan et al. (2024)

| Model | Prompt-level Acc. (strict) | Prompt-level Acc. (loose) | Instruction-level Acc. (loose) |
|---|---|---|---|
| Minitron 4B-instruct | **68.76** | **73.01** | **81.29** |
| Gemma-2B-IT | - | 28.70 | 40.50 |
| Qwen2-1.5B-Instruct | 29 | - | - |

Table 8: Evaluation results on IFEval. Source: Muralidharan et al. (2024)

| Model | Avg |
|---|---|
| Minitron 4B-instruct | **41.11** |
| Gemma-2B-IT | 33.31 |

Table 9: Evaluation results on ChatRAG-Bench. Source: Muralidharan et al. (2024)

| Model | Avg. |
|---|---|
| Minitron 4B-instruct | **53.09** |
| Gemma-2B-IT | 41.63 |
| Llama-3-8B-instruct | 50.51 |

Table 10: Evaluation results on BFCL v2. Source: Muralidharan et al. (2024)

### H.3 Architectural Specifications for Nemotron-Minitron Family

| Model | Layers | Hidden Size | Att. Heads | Query Groups | MLP Hidden | Parameters |
|---|---|---|---|---|---|---|
| Nemotron-4 15B | 32 | 6144 | 48 | 8 | 24576 | 15.6B |
| Nemotron-3 8B | 32 | 4096 | 32 | 32 | 16384 | 8.5B |
| Minitron 8B | 32 | 4096 | 48 | 8 | 16384 | 8.27B |
| Minitron 4B | 32 | 3072 | 24 | 8 | 9216 | 4.19B |

Table 11: Architecture details of the Nemotron-Minitron family. Vocabulary size is 256k for all models. Source: Muralidharan et al. (2024)

## References

Bo Adler, Niket Agarwal, Ashwath Aithal, Dong H Anh, Pallab Bhattacharya, Annika Brundyn, Jared Casper, Bryan Catanzaro, Sharon Clay, Jonathan Cohen, et al. Nemotron-4 340b technical report. *arXiv preprint arXiv:2406.11704*, 2024.

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

Akhiad Bercovich, Tomer Ronen, Talor Abramovich, Nir Ailon, Nave Assaf, Mohammed Dabbah, Ido Galil, Amnon Geifman, Yonatan Geifman, Izhak Golan, Netanel Haber, Ehud Dov Karpas, Roi Koren, Itay Levy, Pavlo Molchanov, Shahar Mor, Zach Moshe, Najeeb Nabwani, Omri Puny, Ran Rubin, Itamar Schen, Ido Shahaf, Oren Tropp, Omer Ullman Argov, Ran Zilberstein, and Ran El-Yaniv. Puzzle: Distillation-based NAS for inference-optimized LLMs. In *Forty-second International Conference on Machine Learning*, 2025.

Aaron Blakeman, Aarti Basant, Abhinav Khattar, Adithya Renduchintala, Akhiad Bercovich, Aleksander Ficek, Alexis Bjorlin, Ali Taghibakhshi, Amala Sanjay Deshmukh, Ameya Sunil Mahabaleshwarkar, et al. Nemotron-h: A family of accurate and efficient hybrid mamba-transformer models. *arXiv preprint arXiv:2504.03624*, 2025.

Han Cai, Chuang Gan, Tianzhe Wang, Zhekai Zhang, and Song Han. Once-for-all: Train one network and specialize it for efficient deployment. In *International Conference on Learning Representations*, 2020.

Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):10558–10578, 2024.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.

Xuanyi Dong and Yi Yang. Network pruning via transformable architecture search. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21, 2019.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

G Hinton. Distilling the knowledge in a neural network. In *Deep Learning and Representation Learning Workshop in Conjunction with NIPS*, 2014.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.

Bo-Kyeong Kim, Geonmin Kim, Tae-Ho Kim, Thibault Castells, Shinkook Choi, Junho Shin, and Hyoung-Kyu Song. Shortened LLaMA: A simple depth pruning for large language models. In *ICLR 2024 Workshop on Mathematical and Empirical Understanding of Foundation Models*, 2024.

Aaron Klein, Jacek Golebiowski, Xingchen Ma, Valerio Perrone, and Cedric Archambeau. Structural pruning of pre-trained language models via neural architecture search. *Transactions on Machine Learning Research*, 2024.

Liam Li and Ameet Talwalkar. Random search and reproducibility for neural architecture search. In *Proceedings of The 35th Uncertainty in Artificial Intelligence Conference*, volume 115, pages 367–377. PMLR, 2020.

Zhengang Li, Geng Yuan, Wei Niu, Pu Zhao, Yanyu Li, Yuxuan Cai, Xuan Shen, Zheng Zhan, Zhenglun Kong, Qing Jin, Zhiyu Chen, Sijia Liu, Kaiyuan Yang, Bin Ren, Yanzhi Wang, and Xue Lin. Npas: A compiler-aware framework of unified network pruning and architecture search for beyond real-time mobile acceleration. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14250–14261, 2021.

Pavlo Molchanov, Jimmy Hall, Hongxu Yin, Jan Kautz, Nicolo Fusi, and Arash Vahdat. Lana: Latency aware network acceleration. In *Computer Vision – ECCV 2022*, pages 137–156, Cham, 2022.

Saurav Muralidharan, Sharath Turuvekere Sreenivas, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov. Compact language models via pruning and knowledge distillation. *Advances in Neural Information Processing Systems*, 37:41076–41102, 2024.

Jupinder Parmar, Shrimai Prabhumoye, Joseph Jennings, Mostofa Patwary, Sandeep Subramanian, Dan Su, Chen Zhu, Deepak Narayanan, Aastha Jhunjhunwala, Ayush Dattagupta, et al. Nemotron-4 15b technical report. *arXiv preprint arXiv:2402.16819*, 2024.

Sharath Turuvekere Sreenivas, Saurav Muralidharan, Raviraj Joshi, Marcin Chochowski, Ameya Sunil Mahabaleshwarkar, Gerald Shen, Jiaqi Zeng, Zijia Chen, Yoshi Suhara,

Shizhe Diao, et al. Llm pruning and distillation in practice: The minitron approach. *arXiv preprint arXiv:2408.11796*, 2024.

Rhea Sanjay Sukthanker, Benedikt Staffler, Frank Hutter, and Aaron Klein. Compressing large language models with automated sub-network search. *arXiv preprint arXiv:2410.06479*, 2024.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Fali Wang, Zhiwei Zhang, Xianren Zhang, Zongyu Wu, Tzuhao Mo, Qiuhao Lu, Wanjing Wang, Rui Li, Junjie Xu, Xianfeng Tang, et al. A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with llms, and trustworthiness. *arXiv preprint arXiv:2411.03350*, 2024a.

Wenxiao Wang, Wei Chen, Yicong Luo, Yongliu Long, Zhengkai Lin, Liye Zhang, Binbin Lin, Deng Cai, and Xiaofei He. Model compression and efficient inference for large language models: A survey. *arXiv preprint arXiv:2402.09748*, 2024b.

Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. Sheared LLaMA: Accelerating language model pre-training via structured pruning. In *The Twelfth International Conference on Learning Representations*, 2024.

Yifei Yang, Zouying Cao, and Hai Zhao. Laco: Large language model pruning via layer collapse. *arXiv preprint arXiv:2402.11187*, 2024.

Jiahui Yu and Thomas Huang. Universally slimmable networks and improved training techniques. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1803–1811, 2019.

Jiahui Yu, Pengchong Jin, Hanxiao Liu, Gabriel Bender, Pieter-Jan Kindermans, Mingxing Tan, Thomas Huang, Xiaodan Song, Ruoming Pang, and Quoc Le. Bignas: Scaling up neural architecture search with big single-stage models. In *Computer Vision – ECCV 2020*, pages 702–717, Cham, 2020.